

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

Направление	09.03.04 - Программная инженерия
Профиль	Разработка программно-информационных систем
Факультет	КТИ
Кафедра	МО ЭВМ

К защите допустить

Зав. кафедрой

А.А. Лисс

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
БАКАЛАВРА**

**Тема: РАЗРАБОТКА МОДЕЛИ ИСКУССТВЕННОГО ИНТЕЛЛЕКТА
ДЛЯ ПРОГНОЗИРОВАНИЯ ПОВЕДЕНИЯ ПРОГРАММ В ОС**

Студент			П.А. Бодунов
		<i>подпись</i>	
Руководитель	доцент, к.т.н.		А.А. Лисс
		<i>подпись</i>	
Консультанты			А.В. Гаврилов
		<i>подпись</i>	
	к.т.н.		М.М. Заславский
		<i>подпись</i>	
			А.И. Маловский
		<i>подпись</i>	

Санкт-Петербург
2024

ЗАДАНИЕ

НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ

Утверждаю

Зав. кафедрой МО ЭВМ

_____ А.А. Лисс

« » 2024 г.

Студент Бодунов П.А.

Группа 0303

Тема работы: Разработка модели искусственного интеллекта для прогнозирования поведения программ в ОС

Место выполнения ВКР: СПбГЭТУ «ЛЭТИ», кафедра МО ЭВМ

Исходные данные (технические требования):

Необходимо разработать модель искусственного интеллекта для предсказания следующего системного вызова для конкретной программы.

Содержание ВКР:

Введение, Обзор предметной области, Формулировка требований к решению и постановка задачи, Проектирование модели ИИ, Реализация модели ИИ, Заключение

Перечень отчетных материалов: пояснительная записка, иллюстративный материал

Дополнительные разделы: безопасность жизнедеятельности

Дата выдачи задания

Дата представления ВКР к защите

«02» апреля 2024 г.

«05» ИЮНЯ 2024 Г.

Студент

П.А. Бодунов

Руководитель доцент, к.т.н.

А.А. Лисс

Консультант

А.В. Гаврилов

КАЛЕНДАРНЫЙ ПЛАН ВЫПОЛНЕНИЯ ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЫ

Утверждаю

Зав. кафедрой МО ЭВМ

— А.А. Лисс

« 20 Г.

Студент Бодунов П.А.

Группа 0303

Тема работы: Разработка модели искусственного интеллекта для прогнозирования поведения программ в ОС

№ п/п	Наименование работ	Срок выполнения
1	Обзор литературы по теме работы	17.11 – 30.01
2	Анализ требований и проектирование приложения	30.01 – 01.02
3	Разработка приложения	01.02 – 15.04
5	Оформление пояснительной записки	15.04 – 07.05
6	Оформление иллюстративного материала	07.05 – 12.05
7	Предзащита	30.05.2024

Студент П.А. Бодунов

Руководитель доцент, к.т.н. А.А. Лисс

Консультант А.В. Гаврилов

РЕФЕРАТ

Пояснительная записка 52 стр., 19 рис., 2 табл., 40 ист.

НЕЙРОННЫЕ СЕТИ, ТРАССИРОВКА, СИСТЕМНЫЕ ВЫЗОВЫ,
ПРЕДСКАЗАНИЕ СИСТЕМНЫХ ВЫЗОВОВ

Объектом исследования является модель прогнозирования системных вызовов.

Предметом исследования является точность и время предсказания системных вызовов.

Цель работы: разработка модели искусственного интеллекта для прогнозирования действий программ для улучшения производительности операционных систем.

В данной выпускной квалификационной работе изучаются подходы к предсказанию поведения программ с целью выявления наиболее эффективного метода для прогнозирования системных вызовов. Знание поведения программ полезно при разработке более совершенных методов планирования процессов в ОС. Проведено исследование различных аналогов для модели прогнозирования. Были рассмотрены сверточные нейронные сети CNN, а также рекуррентные нейронные сети: GRU, LSTM, Bidirectional LSTM (Bi-LSTM). Нейронная сеть, состоящая из слоев: Word Embedding, Bi-LSTM и линейного, является более точным методом прогнозирования по сравнению с другими методами, поскольку он способен учитывать и использовать информацию как о предыдущих, так и о последующих временных шагах, а также учитывает последовательность слов. Это позволяет модели более полно и точно анализировать зависимости в данных и делать более точные прогнозы. Исследование данного метода показало, что он недостаточно хорош за счет своей долгой работы, поэтому было принято решение использовать нейронную сеть со слоями: Word Embedding и линейным. Данный метод показал допустимое время прогнозирования для использования в ОС.

ABSTRACT

In this final qualifying work, approaches to predicting the behavior of programs are studied in order to identify the most effective method for predicting system challenges. Knowledge of program behavior is useful in developing better methods for planning processes in the OS. A study of various analogues for the forecasting model has been conducted. CNN convolutional neural networks were considered, as well as recurrent neural networks: GRU, LSTM, Bidirectional LSTM (Bi-LSTM). A neural network consisting of layers: Word Embedding, Bi-LSTM and linear is a more accurate forecasting method compared to other methods, since it is able to take into account and use information about both previous and subsequent time steps, and also takes into account the sequence of words. This allows the model to more fully and accurately analyze the dependencies in the data and make more accurate predictions. The study of this method showed that it is not good enough due to its long work, so it was decided to use a neural network with layers: Word Embedding and linear. This method showed the acceptable prediction time for use in the OS.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	10
1 Обзор предметной области	12
1.1 Роль системных вызовов в ОС	12
1.2 Существующие модели предсказания.....	14
1.2.1 LSTM.....	14
1.2.2 GRU	17
1.2.3 CNN.....	19
1.2.4 Bi-LSTM.....	20
1.3 Критерии сравнения аналогов.....	21
1.3.1 Точность прогнозирования.....	21
1.3.2 Алгоритмическая сложность нейронной сети.....	22
1.3.3 Выводы по итогам сравнений	22
2 Формулировка требований к решению и постановка задачи	24
2.1 Постановка задачи.....	24
2.2 Требования к решению	24
2.3 Обоснование требований к решению	24
2.4 Обоснование выбора метода решения	25
3 Проектирование модели ИИ.....	26
3.1 Формирование датасета	26
3.1.1 Сбор данных.....	26
3.1.2 Токенизация	26
3.2 Архитектура и принцип работы Нейронной сети.....	27
3.2.1 Word Embedding.....	27
3.2.2 Линейный слой	28
3.2.3 Спроектированная архитектура Нейронной сети	29
4 Реализация модели ИИ.....	31
4.1 Датасет.....	31
4.1.1 Сбор датасета	31
4.1.2 Dataset и dataloader	32
4.2 Реализация нейронной сети и ее обучение	33
4.3 Используемые технологии	38
5 БЕЗОПАСНОСТЬ ЖИЗНЕДЕЯТЕЛЬНОСТИ.....	40
5.1 Требования к производственным помещениям	40
5.2 Требования к рабочему месту сотрудника	41

5.3 Эргономика интерактивной системы	42
5.4 Оценка используемого ПО	45
5.5 Выводы	46
ЗАКЛЮЧЕНИЕ	47
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	49

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

Датасет (выборка) – набор данных.

ЯП – язык программирования.

IDE – среда разработки программного обеспечения, которая предоставляет разработчикам инструменты для создания, отладки и тестирования текста программы.

Прикладные программы – это программы, предназначенные для выполнения определенных задач и взаимодействия с пользователями.

Операционная система (ОС) – это программное обеспечение, которое управляет аппаратными ресурсами компьютера и обеспечивает работу других прикладных программ.

Процесс в Linux – это программа, которая выполняется в операционной системе.

Трассировка системных вызовов – это процесс отслеживания и записи всех системных вызовов, которые выполняются программой во время ее работы.

Искусственный нейрон – это математическая модель биологического нейрона.

Искусственный интеллект (ИИ) – это технология, которая позволяет компьютерным программам анализировать данные, обучаться и самостоятельно решать задачи, сходные со способностями человеческого интеллекта.

Нейронный слой – это совокупность нейронов, выполняющая определенные функции при обработке и передаче данных от одного слоя к другому.

Искусственная нейронная сеть (нейронная сеть) – это математическая модель, состоящая из набора связанных между собой нейронных слоев и созданная на основе функционирования биологических нейронных сетей в организме человека.

Эпоха – это период времени, в течение которого нейронная сеть обучается на данных, обновляя свои веса и повышая точность модели.

Математическая лингвистика – это область науки, которая изучает методы математики и статистики для анализа текстов, генерации и понимания языка, распознавания речи и других задач, связанных с лингвистикой.

Обработка естественного языка (NLP) – это область математической лингвистики и искусственного интеллекта, которая занимается обработкой и анализом человеческого языка.

Токенизация – это процесс разделения текста на токены.

Токен – это единица обработки текста для задачи NLP.

Word Embedding – это преобразование текстовой информации в числовой формат (числовые векторы в многомерном пространстве), который помогает модели эффективнее решать задачу NLP.

Рекуррентные нейронные сети (RNN) – это вид нейронных сетей, образующие направленную последовательность связей между элементами и сохраняющие информацию о предыдущих состояниях для принятия решений на основе текущих входных данных.

ВВЕДЕНИЕ

Современные операционные системы сталкиваются со стремительно растущим объемом данных и вычислительной сложностью работы приложений, что приводит к снижению производительности и уменьшению быстродействия системы. Для решения этой проблемы разрабатываются специальные алгоритмы, которые позволяют предсказывать поведение программ. С помощью предсказания поведения программ ОС может более эффективно выделять ресурсы, предотвращать ситуации, связанные с нехваткой ресурсов, назначать приоритеты для выполнения программ. Таким образом, предсказание поведения программ позволяет оптимизировать производительность операционной системы за счет предварительной загрузки необходимых данных и программ в память, уменьшения загрузки центрального процессора (CPU), более эффективного использования памяти, повышения энергоэффективности и сокращения времени ожидания выполнения системных вызовов [1], [2], [5].

Целью работы является разработка модели искусственного интеллекта для прогнозирования действий программ, способствуя улучшению производительности операционных систем.

Для достижения поставленной цели необходимо выполнить следующие задачи:

- Изучить существующие решения для предсказания последующего системного вызова;
- Составить список требований к решению;
- Реализовать трассировку списка процессов для получения списка последовательных событий (системных вызовов);
- Обработать список системных вызовов и разделить полученные данные на обучающую, валидационную и тестовую выборки;
- Выбрать подходящую модель ИИ для прогнозирования системных вызовов;
- Провести обучение модели на обучающем наборе данных;
- Оценить качество модели ИИ на тестовой выборке.

Объектом исследования является модель прогнозирования системных вызовов в ОС Linux.

Предметом исследования является точность и время предсказания системных вызовов.

Разработка модели искусственного интеллекта для прогнозирования поведения программ в операционных системах имеет практическую значимость и может найти применение в таких областях, как информационная безопасность, оптимизация работы программ и повышения эффективности IT-инфраструктуры.

1 Обзор предметной области

1.1 Роль системных вызовов в ОС

Операционная система Linux – одна из наиболее широко используемых операционных систем [7], относящаяся к семейству Unix-подобных операционных систем, основанных на ядре Linux, с открытым исходным текстом программы. Система состоит из следующих основных компонентов [39]:

1. Ядро Linux – программа, лежащая в основе операционной системы, управляющая работой аппаратного обеспечения, выполняющая основные функции операционной системы и отвечающая за выполнение прикладных программ;
2. Командная оболочка (shell) – основанный на текстовых командах интерфейс, с помощью которого пользователи взаимодействуют с операционной системой;
3. Системные утилиты – это набор базовых программ, который обеспечивает выполнение основных функций операционной системы;
4. Графическое окружение – набор программ и библиотек, обеспечивающий графический интерфейс пользователя (GUI) для удобства работы с операционной системой;
5. Драйверы устройств – программное обеспечение, позволяющее операционной системе взаимодействовать с аппаратным обеспечением компьютера.

Для обеспечения стабильной и эффективной работы операционной системы Linux эти компоненты взаимодействуют друг с другом. Операционная система обладает двумя режимами работы: пользовательским режимом и режимом ядра [40]:

- Пользовательский режим – это режим работы, в котором выполняются пользовательские процессы или пользовательские программы. В этом режиме процессы имеют ограниченный доступ к ресурсам компьютера и не имеют прямого доступа к аппаратному

обеспечению. Пользовательский режим обеспечивает изоляцию процессов друг от друга и обеспечивает безопасность системы.

- Режим ядра – это режим работы, в котором выполняется только ядро операционной системы. В данном режиме ядро ОС имеет полный контроль над ресурсами компьютера и осуществляет их управление. Ядро операционной системы управляет обработкой прерываний, переключением контекста процессов, управлением памятью и другими низкоуровневыми операциями.

Пользовательский режим взаимодействует с режимом ядра для обеспечения работы операционной системы. Ядро операционной системы предоставляет интерфейс для взаимодействия между пользовательскими программами и аппаратным обеспечением компьютера, обеспечивает безопасность и стабильность работы системы.

Пространство пользователя может взаимодействовать с пространством ядра посредством системных вызовов.

Системный вызов [39] – это функция операционной системы, которая позволяет приложению или пользовательскому процессу взаимодействовать с ядром операционной системы и получать доступ к различным ресурсам и функциям системы. Системные вызовы обеспечивают интерфейс для работы с аппаратным обеспечением компьютера через операционную систему.

Состояние процесса в конкретный момент времени называется контекстом.

Переключение контекста (context switch) [8] – это процесс, при котором операционная система переключает выполнение процессора с выполнения одного процесса на другой. При переключении контекста выполнение текущего процесса останавливается и его контекст сохраняется, чтобы при возобновлении процесса он продолжил работу с того же момента, на котором был прерван.

Стоимость контекстного переключения (context switch) зависит от множества факторов и считается затратной операцией [8], так как требует сохранения и загрузки состояния процесса, включая значения регистров, указатели стека, таблицы страниц памяти и другие данные. Уменьшение частоты переключения контекста является ключевым аспектом повышения производительности системы.

Предсказание следующего системного вызова позволяет заранее загружать необходимые данные в память и выбирать оптимальный момент переключения контекста, что приводит к сокращению времени ожидания выполнения системного вызова и уменьшению частоты переключения контекста. Таким образом повышается производительность операционной системы.

1.2 Существующие модели предсказания

1.2.1 LSTM

LSTM (Long Short-Term Memory) [9], [10], [11], [12] – это разновидность архитектуры рекуррентных нейронных сетей, которая способна запоминать информацию на длительные промежутки времени. LSTM состоит из специальных блоков памяти, которые позволяют сети запоминать и забывать информацию в зависимости от контекста.

Архитектура LSTM представлена на рис. 1, где:

- Neural node – это обученный слой нейронной сети;
- Pointwise operation – поточечная операция;
- Vector transfer – векторный перенос (перемещение вектора с выхода одного узла на вход другого);
- Concatenate – объединение параметров, подающихся на вход слою или операции;
- Copy – копирование (информация копируется и отправляется в различные узлы сети);
- Bias – добавление смещения.

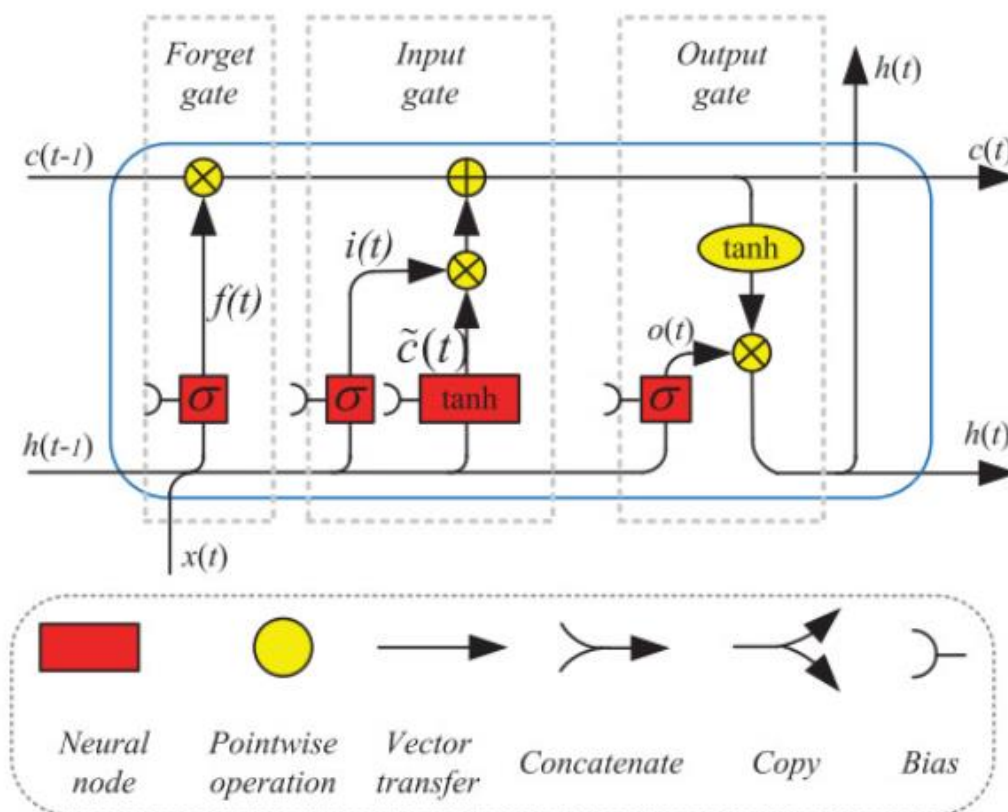


Рисунок 1 – Архитектура LSTM

В блоке памяти LSTM существует несколько основных «фильтров» [14]:

1. Forget gate определяет, какую информацию можно забыть из предыдущей ячейки памяти, сохраняет только актуальную информацию;
2. Input gate решает, какую информацию следует добавить в ячейку памяти на основе нового входного сигнала, и позволяет модели обновлять свою память с учетом новых данных;
3. Output gate определяет, какую информацию из ячейки памяти следует передать на выход.

Математически LSTM выражена следующим образом (см. рис. 2), где

- x_t – входной вектор;
- h_t – выходной вектор текущего блока;
- σ – функция активации сигмоида;
- c_t – состояние текущего блока;
- W_f, W_i, W_c, W_o – матрицы параметров блока;

- b_f, b_i, b_c, b_o – векторы параметров блока;
- f_t – вектор запоминания старой информации;
- i_t – вектор получения новой информации;
- $\tilde{c}_t$ – вектор значений, который можно добавить в состояние ячейки;
- o_t – вектор информации, которая будет выведена из состояния ячейки.

$$\begin{aligned}
f_t &= \sigma(W_{fh}h_{t-1} + W_{fx}x_t + b_f), \\
i_t &= \sigma(W_{ih}h_{t-1} + W_{ix}x_t + b_i), \\
\tilde{c}_t &= \tanh(W_{ch}h_{t-1} + W_{cx}x_t + b_c), \\
c_t &= f_t \cdot c_{t-1} + i_t \cdot \tilde{c}_t, \\
o_t &= \sigma(W_{oh}h_{t-1} + W_{ox}x_t + b_o), \\
h_t &= o_t \cdot \tanh(c_t).
\end{aligned}$$

Рисунок 2 – Математическое представление LSTM блока

Сложность одного скрытого состояния равна:

$O(h \cdot N)$, (1) где

- h – количество скрытых нейронов;
- N – максимум из h и размерности входного вектора токена d .

Произведение матрицы размера $n \times m$ на вектор m дает сложность $O(n \cdot m)$. Сложение двух векторов требует $O(h)$ вычислений. Применение σ и $\tanh$ требует $O(h)$ операций.

Произведение $W_{fh}h_{t-1}$, $W_{fx}x_t$ матриц на вектора и требует $O(hd)$ и $O(h^2)$ операций соответственно, сложение с вектором b_f и применение сигмоиды σ требует по $O(h)$ операций. Поэтому сложность f_t равна:

$$O(hd + h^2 + h + h) = O(h(d + h + 2)), (2)$$

$i_t, \tilde{c}_t, o_t$ требуют такое же количество вычислений, как f_t (формула (2)).

Вычисления c_t и h_t требует по $O(2h)$ операций. Итоговая сложность вычислений скрытого состояния равна:

$$O(4h(d + h + 2) + 4h) = O(4h(d + h + 3)), (3)$$

Отбросив константы и несущественную часть, сложность одного скрытого состояния будет равна $O(hN)$. Сложность всего LSTM блока равна:

$$O(4hT(d + h + 3)) \approx O(h \cdot T \cdot N), (4)$$

где T – количество токенов в предложении.

LSTM слои широко применяются для анализа и прогнозирования последовательных данных в различных областях науки и техники благодаря их способности эффективно работать с долгосрочными зависимостями в данных [15].

1.2.2 GRU

GRU (Gated Recurrent Unit) [14], [33], [34] – это упрощенная версия LSTM, которая также использует фильтры для управления потоком информации. Forget gate и input gate объединяются в один фильтр update gate. Этот фильтр определяет сколько информации сохранить от последнего состояния, и сколько информации получить от предыдущего слоя. Фильтр reset gate работает почти так же, как фильтр forget gate.

Математически GRU выражена следующим образом (см. рис. 3):

$$\begin{aligned} r_t &= \sigma(W_{rh}h_{t-1} + W_{rx}x_t + b_r), \\ z_t &= \sigma(W_{zh}h_{t-1} + W_{zx}x_t + b_z), \\ \tilde{h}_t &= \tanh(W_{\tilde{h}h}(r_t \cdot h_{t-1}) + W_{\tilde{h}x}x_t + b_{\tilde{h}}), \\ h_t &= (1 - z_t) \cdot h_{t-1} + z_t \cdot \tilde{h}_t. \end{aligned}$$

Рисунок 3 – Математическое представление GRU блока

Архитектура GRU представлена на рис. 4, где:

- x_t – входной вектор;
- h_t – выходной вектор текущего блока;

- σ – функция активации сигмоида;
- $W_r, W_i, W_{\tilde{h}}$ – матрицы параметров блока;
- $b_r, b_i, b_{\tilde{h}}$ – векторы параметров блока;
- r_t – вектор информации, которую необходимо сбросить;
- z_t – вектор информации, которую необходимо обновить;
- $\tilde{h}_t$ – вектор значений, которые можно добавить на выход;

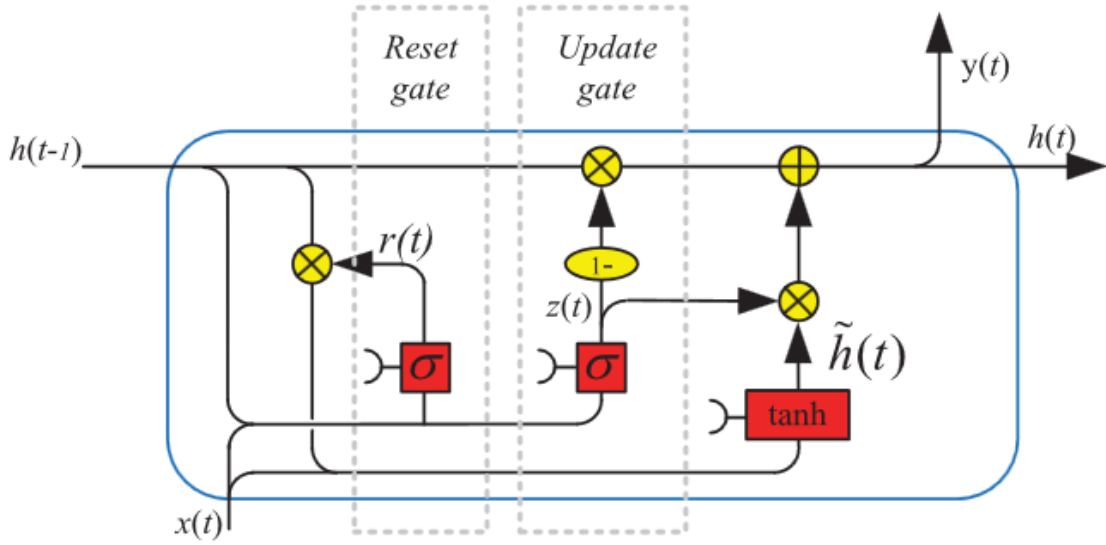


Рисунок 4 – Архитектура блока GRU

Аналогично LSTM сложность одного скрытого состояния GRU равна $O(hN)$, где h – длина последовательности (количество токенов), N – максимум из h и размерности вектора токена d .

Аналогично f_t в LSTM сложность вычисления r_t, z_t , как в формуле (2). Перемножение векторов r_t и h_{t-1} требует $O(h)$ операций, поэтому вычисления $\tilde{h}$ требует:

$$O(h + h^2 + hd + h + h) = O(h(h + d + 3)), (5)$$

h_t требует $O(2h)$ операций, поэтому итоговая сложность вычислений скрытого состояния равна

$$O(2h(d + h + 2) + h(h + d + 3) + 2h) = O(3h(h + d + 3)), (6)$$

Отбросив константы и несущественную часть, сложность одного скрытого состояния будет равна $O(hN)$. Сложность всего GRU блока равна:

$$O(3hT(d + h + 3)) \approx O(h \cdot T \cdot N), (7)$$

где T – количество токенов в предложении.

1.2.3 CNN

Свертка – это математическая операция, используемая для извлечения признаков из входных данных, таких как изображения или текст.

Сверточная нейронная сеть (CNN) [3], [30], [31] – это вид нейронных сетей, использующий свертки. Основная архитектура CNN для предсказания текста включает в себя следующие слои [3]:

1. Входной слой – набор векторов, каждый из которых соответствует одному токену во входном тексте.
2. Сверточный слой применяет набор фильтров (ядер) ко входным данным для извлечения локальных объектов. Результатом работы сверточного слоя является набор карт признаков объектов.
3. Пулинговый слой используется для уменьшения размера карт объектов, полученных из сверточного слоя, с помощью максимального (выбор максимальных значения из сверточного слоя) или среднего пулинга (взятие среднего). Пулинговый слой уменьшает вычислительную сложность модели и улучшает ее способность обобщать информацию.
4. Выходной слой – слой, который содержит нейроны, представляющие результат работы модели искусственного интеллекта.

Архитектура CNN для обработки текста представлена на рис. 5:

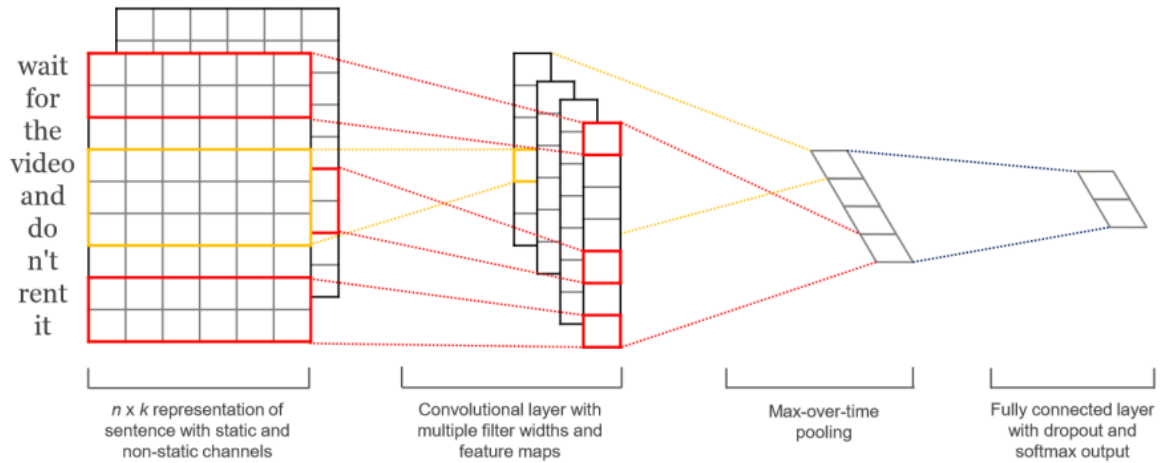


Рисунок 5 – Архитектура CNN для обработки текста

Сложность для CNN равна:

$$O(\sum_{i=1}^F (T - K_i + 1) \cdot d \cdot K_i), \quad (8) \text{ где}$$

- d – размерность вектора токена;
- T – количество токенов на входе;
- K_i – размер ядра i -го признака;
- F – количество карт признаков.

Сложность свертки одного элемента равна $O(d \cdot K_i)$. Ядро проходит все предложение за $T - K_i + 1$ операций, поэтому сложность вычисления одной карты признаков равна:

$$O((T - K_i + 1) \cdot d \cdot K_i), \quad (9)$$

Количество карт признаков F штук, поэтому итоговая сложность CNN равна:

$$O(\sum_{i=1}^F (T - K_i + 1) \cdot d \cdot K_i) \approx O(d \cdot T \cdot \sum_{i=1}^F K_i), \quad (10)$$

1.2.4 Bi-LSTM

Сеть с двунаправленной LSTM (Bidirectional LSTM Network, Bi-LSTM) [13], [14], [32] – тип рекуррентной нейронной сети, который использует LSTM блоки и обрабатывает входные данные как в прямом, так и в обратном направлении. Этот подход позволяет учитывать, как предыдущий, так и последующий контекст для более точного прогнозирования выходных данных.

Структура Bi-LSTM представлена на рис. 6:

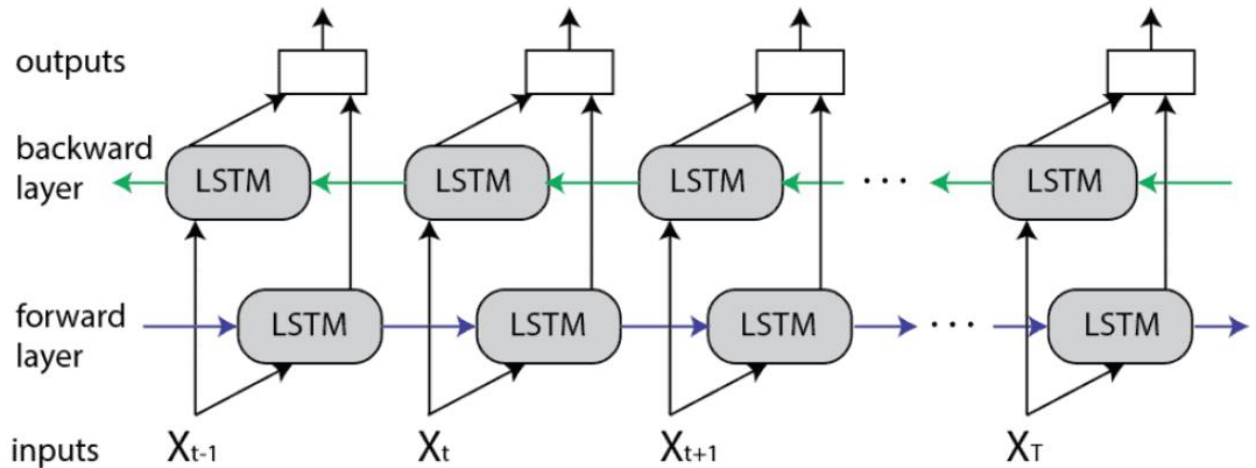


Рисунок 6 – Структура Bi-LSTM

Bi-LSTM состоит из двух слоев [14]:

- Forward layer (прямой слой) работает в прямом направлении, начиная с первого элемента последовательности и последовательно обрабатывает следующие элементы до последнего;
- Backward layer (обратный слой) работает в обратном направлении, начиная с последнего элемента последовательности, и последовательно обрабатывает предыдущие элементы до первого.

Данный метод позволяет модели лучше прогнозировать следующий элемент на основе предыдущего и будущего контекста, учитывая порядок слов.

Сложность Bidirectional LSTM отличается от LSTM в 2 раза, т. к. в Bi-LSTM 2 слоя: Forward layer и Backward layer. Поэтому сложность Bi-LSTM равна формуле (4) умноженной на 2:

$$O(8hT(d + h + 3)) \approx O(h \cdot T \cdot N), (11)$$

1.3 Критерии сравнения аналогов

1.3.1 Точность прогнозирования

Точность прогнозирования системных вызовов является важным фактором, влияющим на эффективность оптимизации производительности операционной системы. Точность зависит от используемых алгоритмов прогнозирования, объема доступных данных и качества обучающей выборки. Для оценки

точности прогнозирования данные разделяются на обучающую, валидационную и тестовую выборки. Обучающая выборка необходима для обучения модели искусственного интеллекта, валидационная – для оценки модели во время обучения, а тестовая – для проверки обученной модели искусственного интеллекта. Одним из распространенных методов оценки точности прогнозирования является процент правильно предсказанных системных вызовов относительно общего числа вызовов.

1.3.2 Алгоритмическая сложность нейронной сети

Алгоритмическая сложность по времени нейронной сети определяется количеством операций, которые необходимо выполнить для обработки входных данных. Одним из основных способов оценки алгоритмической сложности нейронной сети является асимптотический анализ. Он позволяет оценить сложность нейронной сети при больших размерах входных данных, то есть при стремлении размера входных данных к бесконечности. Асимптотический анализ использует нотацию "О-большое", который позволяет оценить сложность по наиболее значимому члену.

Сравнительная характеристика аналогов представлена в таблице 1:

Таблица 1 – Сравнение аналогов

Метод	Точность	Сложность
CNN	74%	$O(\sum_{i=1}^F (T - K_i + 1) \cdot d \cdot K_i) \approx O(d \cdot T \cdot \sum_{i=1}^F K_i)$
GRU	78%	$O(3hT(d + h + 3)) \approx O(h \cdot T \cdot N)$
LSTM	77%	$O(4hT(d + h + 3)) \approx O(h \cdot T \cdot N)$
Bi-LSTM	>77%	$O(8hT(d + h + 3)) \approx O(h \cdot T \cdot N)$

1.3.3 Выводы по итогам сравнений

Для сравнения были выбраны лучшие модели LSTM, GRU и CNN, которые были приведены в статье [4].

Таким образом были выбраны модели CNN [27], LSTM [28], GRU [29] точности были вычислены, как среднее арифметическое точностей прогнозирования на данных: Helpdesk, BPI 2012, BPI 2012 Complete, BPI 2012 W, BPI 2012 W Complete, BPI 2012 O, BPI 2012 A, BPI 2013 closed problems, BPI 2013 Incidents, Env permit. Точность CNN чуть меньше, чем точность LSTM и GRU, а точности LSTM и GRU почти одинаковые (разница в 1%). Сравнение методов происходит на данных предсказания событий, потому что прогнозирование системных вызовов можно назвать подзадачей прогнозирования событий.

Таблица показывает приблизительные оценки сложности. Фактическая сложность может зависеть от конкретной реализации и оптимизаций. CNN для текста имеет меньшую арифметическую сложность по сравнению с рекуррентными нейронными сетями, т. к. она использует меньшее количество операций для обработки текстовых данных. CNN использует свертку для извлечения характеристик из текста, что позволяет ей делать это без учёта порядка слов, поэтому данный подход более эффективен с вычислительной точки зрения, чем GRU и LSTM, которым необходимо учитывать последовательность слов при обработке текста.

2 Формулировка требований к решению и постановка задачи

2.1 Постановка задачи

Необходимо собрать датасет системных вызовов, разработать модель искусственного интеллекта для прогнозирования следующего системного вызова по пяти вызовам и оценить точность и время предсказания. Данная задача заключается в классификации системных вызовов и обработке текста, содержащий информацию о системных вызовах, с использованием методов обработки естественного языка (NLP).

2.2 Требования к решению

На основе сранения аналогов решение должно удовлетворять следующим условиям:

1. Модель прогнозирования должна обладать повышенной точностью прогнозирования;
2. Обучение модели на датасете системных вызовов, содержащих информацию о контексте, включая указание на используемое приложение и выполняемое действие с аргументами и результатом выполнения;
3. Обеспечение более точной работы обученной нейронной сети за счет большого количества уникальных данных.

2.3 Обоснование требований к решению

1. Модель ИИ должна обладать точностью прогнозирования не меньше, чем на рассмотренных решениях;
2. Обучение модели должно происходить на данных системных вызовов, содержащих информацию о контексте, включающего указание на используемое приложение и выполняемое действия для более точного предсказания;
3. Модель должна быть обучена на большом датасете для большего охвата уникальных данных для улучшения точности предсказания системных вызовов.

2.4 Обоснование выбора метода решения

Для выполнения условия повышенной точности выбрана нейронная сеть, состоящая из слоев Word Embedding, Bi-LSTM, и линейного. Данный выбор обусловлен способностью данной нейронной сети работать с текстовыми данными и извлекать сложные нелинейные зависимости между входными и выходными данными. Слой Word Embedding позволяет преобразовать текстовую информацию в векторное представление, что значительно упрощает работу с данными в формате текста. Слой Bi-LSTM обладает точностью не хуже, чем у рассмотренных методов, и обеспечивает обработку входной последовательности в обоих направлениях, учитывая порядок слов, что позволяет лучше анализировать контекст и взаимосвязи между различными токенами в тексте. Линейный слой в конце нейронной сети позволяет преобразовывать выходные данные из Bi-LSTM в финальное предсказание системного вызова.

3 Проектирование модели ИИ

3.1 Формирование датасета

3.1.1 Сбор данных

Для обучения нейронных сетей необходимо иметь данные о системных вызовах, которые часто вызываются во время использования компьютера пользователем. Для улучшения точности прогнозирования следует предсказывать название следующего системного вызова на основе анализа нескольких предыдущих системных вызовов.

Для более точного прогнозирования системных вызовов данные должны обладать достаточно большим контекстом и собраны из различных действий пользователя в компьютере (сценариев использования). Контекстом системных вызовов является: название процесса, который вызывает системный вызов, название системного вызова, его аргументы и его результат выполнения.

Наиболее популярные действия пользователя использующего ОС Linux:

- Использование интернет браузеров: посещение сайтов, чтение новостей, просмотр видео и т. д.;
- Использование социальных сетей: чаты, просмотр ленты новостей;
- Открытие и использование различных приложений;
- Использование терминала.

3.1.2 Токенизация

Разделение текста на токены является одним из первых и важных этапов при обработке текстов и для многих методов анализа естественного языка.

В процессе токенизации слов текст разделяется на отдельные слова, символы или фразы в зависимости от конкретной задачи. Токенизация слов может включать в себя удаление знаков препинания, приведение всех слов к нижнему регистру, удаление стоп-слов.

Последовательность из системных вызовов объединяется в предложение, которое следует обработать для нейронной сети, так же необходимо обработать результат нейронной сети (название системного вызова).

В данной задаче токенами будут считаться:

- Название процесса;
- Название системного вызова;
- Полное перечисление аргументов;
- Результат системного вызова.

Пример разбиения данных на токены представлен на рис. 7:

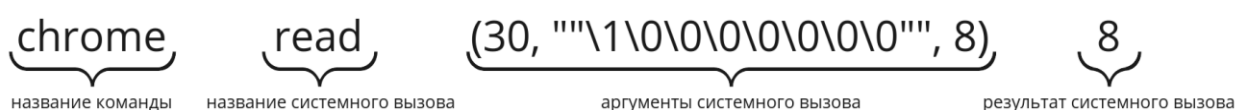


Рисунок 7 – Пример разбиения данных на токены

3.2 Архитектура и принцип работы Нейронной сети

3.2.1 Word Embedding

Для успешного обучения нейронной сети необходимо преобразовать токены в вектора чисел, это можно осуществить при помощи Word Embedding. Вектора, получившиеся после Word Embedding, обладают следующими свойствами [22]:

- Семантическая близость. Слова с похожим значением имеют близкие векторы в пространстве, что позволяет измерять семантическую близость между словами;
- Арифметические операции над векторами слов позволяют находить аналогии.

Данные свойства могут быть полезны из-за существования системных вызовов, которые выполняют задачу с похожим смыслом, но при этом имеют разные названия.

Пример работы Word Embedding представлен на рис. 8, где в левой части изображены токены, а в правой – полученные вектора:

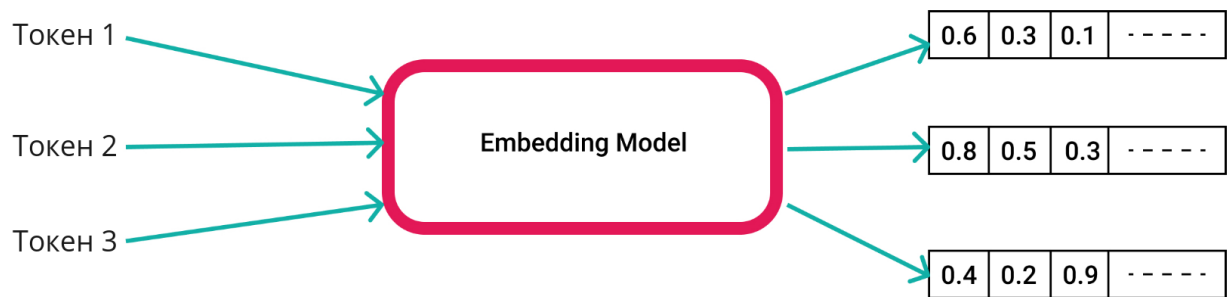


Рисунок 8 – Пример работы Word Embedding

3.2.2 Линейный слой

Линейный слой нейронной сети представляет собой слой, в котором каждый нейрон соединен с каждым нейроном предыдущего слоя, и каждая связь между нейронами имеет свой вес, который определяет влияние сигнала на следующий нейрон.

Линейный слой математически можно представить следующим образом [16] (см. рис. 9), где y – вектор выходных данных, x – вектор входных данных, W – матрица весов (w_{ji} – вес от j -го нейрона к i -ому нейрону), b – вектор сдвига.

$$\begin{pmatrix} y_1 \\ \vdots \\ y_K \end{pmatrix} = \begin{pmatrix} w_{10} & w_{11} & \dots & w_{1D} \\ \vdots & \vdots & \ddots & \vdots \\ w_{K0} & w_{K1} & \dots & w_{KD} \end{pmatrix} \begin{pmatrix} 1 \\ x_1 \\ \vdots \\ x_D \end{pmatrix},$$

$$\mathbf{y} = \mathbf{W}\dot{\mathbf{x}},$$

Рисунок 9 – Математическое представление линейного слоя

Линейный слой нейронной сети представлен на рис. 10:

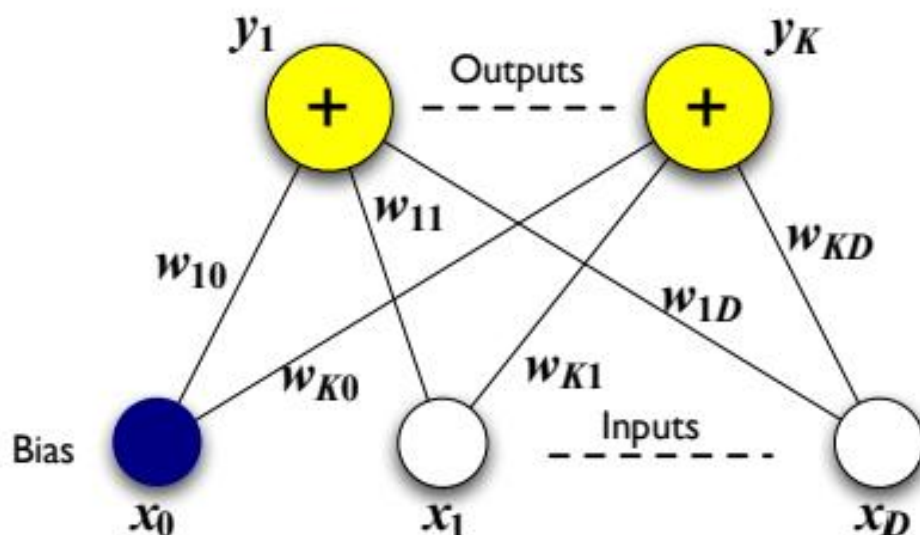


Рисунок 10 – Линейный слой нейронной сети

3.2.3 Спроектированная архитектура Нейронной сети

Текстовые данные системных вызовов токенизируются, токенами являются: название процесса, название системного вызова, полное перечисление его аргументов и результат системного вызова. Далее токены передаются на вход нейронной сети.

Итоговый вид спроектированной нейронной сети состоит из слоя Word Embedding, Bi-LSTM слоя и линейного слоя.

Для преобразования текстовых данных, содержащих информацию о системных вызовах, в векторное представление был использован слой Word Embedding. Такой подход позволяет каждому уникальному токenu присвоить числовой вектор, что улучшает работу нейронной сети и повышает ее эффективность. Получившиеся векторы токенов подаются на вход Bi-LSTM слою.

Слой Bi-LSTM представляет собой двунаправленную рекуррентную нейронную сеть с долгой краткосрочной памятью (LSTM), которая способна анализировать последовательность слоев в прямом и обратном порядке, что позволяет модели учитывать, как предшествующую, так и последующую информацию при обработке данных. LSTM слой способен запоминать предыдущие состояния и принимать решения на основе этой информации.

Линейный слой в конце нейронной сети позволяет классифицировать финальный прогноз поведения программ в операционной системе. Количество выходов линейного слоя равно количеству уникальных названий системных вызовов, встретившихся в данных. Результатом работы линейного слоя будет системный вызов, полученный с наибольшей вероятностью.

Схема спроектированной нейронной сети представлена на рис. 11:

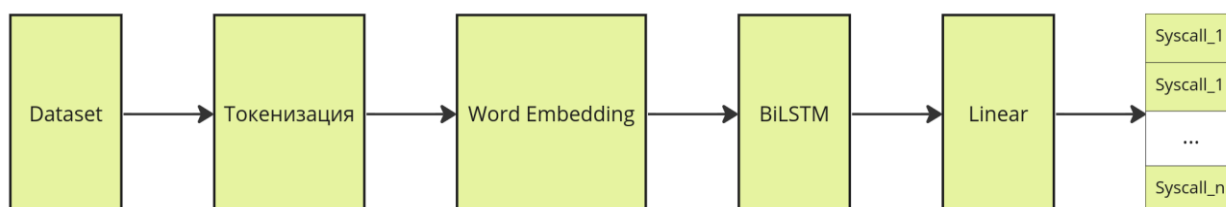


Рисунок 11 – Схема спроектированной нейронной сети

Токенизация входных данных и комбинация слоев Word Embedding, Bi-LSTM, линейного в нейронной сети позволяют эффективно обрабатывать данные и прогнозировать следующие системные вызовы в ОС на основе анализа текстовой информации.

4 Реализация модели ИИ

4.1 Датасет

4.1.1 Сбор датасета

Набор данных формируется с помощью утилиты трассировки `strace`. `Strace` [17] – один из самых популярных инструментов для трассировки системных вызовов. Он позволяет отслеживать все системные вызовы, производимые программой, и выводить подробную информацию о них, такую как тип вызова, аргументы, возвращаемое значение и время выполнения.

Разработана программа, которая устанавливает идентификаторы запущенных процессов в операционной системе, после чего запускает трассировку для каждого процесса с целью отслеживания системных вызовов. Трассировка выполняется при помощи команды `"strace -ff -p pid"`, где `pid` – это уникальный идентификатор запущенного процесса, ключ `-ff` для каждого дочернего процесса создает отдельный файл с именем `"имя_файла.pid"`, `-p` трассирует процесс с заданным `pid`. Таким образом, команда `"strace -ff -p"` будет отслеживать системные вызовы и сигналы нескольких процессов с указанными `pid`. Для того, чтобы в конечном датасете присутствовали названия команд, которые запускали выполнение системных вызовов, название команды добавляется в названия созданных файлов, т.к. в названии команды присутствуют запрещенные символы для названия файла, то использовано регулярное выражение, которое берет название команды до пробела, тем самым заменяя запрещенные символы.

Для прогнозирования будущих системных вызовов было решено использовать данные о пяти предыдущих вызовах. Для составления выборки было решено исключить файлы размером менее 1 килобайта, а также исключить файлы с одинаковыми вызовами, так как они не представляют сильной смысловой ценности. Была разработана программа, которая анализирует все файлы трассировки и отбирает 200 процессов для создания датасета, выбирая их по уникальности названия команды и размеру файла.

Учитывая большой размер и наличие дублирующих данных в полученном датасете, было принято решение выделить 500000 случайно выбранных элементов, оставив из них только уникальные значения. В итоговом датасете получилось 366032 элементов.

4.1.2 Dataset и dataloader

Классы Dataset и DataLoader в PyTorch используются для упрощения и ускорения загрузки и обработки данных во время обучения моделей машинного обучения [18].

Класс Dataset – класс, который предоставляет доступ к набору данных. Он позволяет загружать и обрабатывать данные.

Класс DataLoader используется для создания итератора, который позволяет эффективно загружать данные из созданного объекта класса Dataset во время обучения модели. DataLoader позволяет работать с большими объемами данных и разделять данные на батчи (небольшие подмножества данных из исходного датасета).

Для обработки уже подготовленного датасета были реализованы функции:

1. *tokenizer(syscalls)* – функция на вход получает строку *syscalls*, состоящую из системных вызовов, и токенизирует ее, возвращает список токенов;
2. *add_tokens_to_index(index, tokens_counter)* – преобразование токенов *tokens_counter*, отсортированных по встречаемости, в индексы и добавление токена с индексом в словарь *index*, возвращает словарь *index*.

Был создан класс *SyscallDataset*, наследуемый от класса *Dataset*. В конструкторе данного класса происходит обработка данных токенизация (функцией *tokenizer*) и преобразования токенов в коды (функция *add_tokens_to_index*). Так же были переопределены методы *__len__()*, который возвращает количество элементов в наборе данных, и метод

`__getitem__(ix)`, который возвращает пару (X, y) , где X – тензор индексов токенов, которые были получены путем токенизации предложения, а y – это индекс названия системного вызова.

Также были созданы экземпляры класса *Dataloader*: *train_dataloader*, *val_dataloader*, *test_dataloader*, которые представляют собой данные, разделенные на обучающую, валидационную и тестовую выборки соответственно. Далее на этих объектах нейронная сеть будет обучаться, настраивать свои гиперпараметры и оценивать итоговое качество модели.

4.2 Реализация нейронной сети и ее обучение

Был реализован класс нейронной сети *SyscallBiLSTMNetwork* со следующими слоями:

- Word Embedding, где размерность токена равна 4;
- Bi-LSTM, где используется 1 слой и размер скрытого состояния равен 16;
- Линейный, на вход которого подается 32 выхода за счет двунаправленности слоя LSTM, и возвращает количество выходных значений, равное количеству системных вызовов в наборе данных.

Данные подаются на вход в нейронную сеть батчами по 1024 элемента.

Архитектура получившейся нейронной сети представлена на рис. 12:

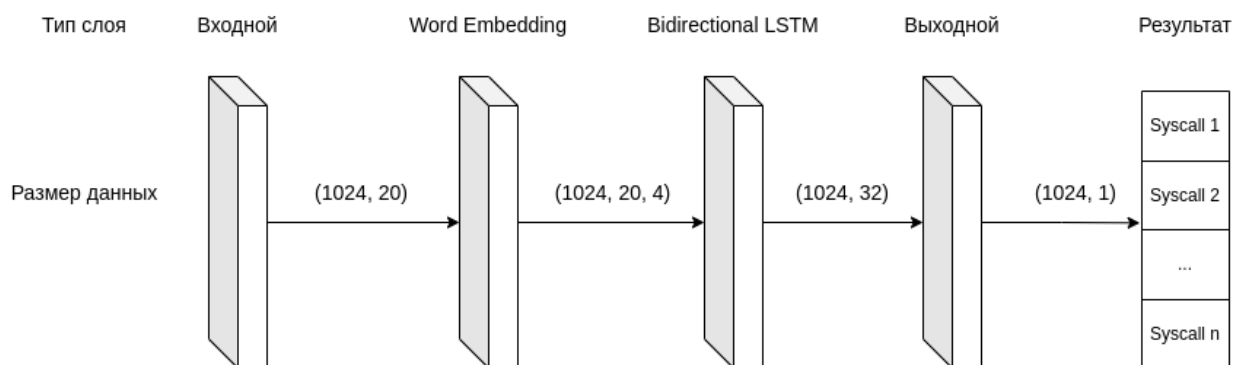


Рисунок 12 – Архитектура получившейся нейронной сети

Так же была реализована функция `compute_loss_accuracy_performance(loss_f, net, dataloader)`, которая вычисляет

функцию потерь (loss), точность и среднее время, которое потребовалось для предсказания последующего вызова, функции потерь *loss_f*, нейронной сети *net* на данных *dataloader*.

В качестве функции потерь была выбрана кросс энтропия (Cross-entropy), т. к. она одна из самых популярных и часто используемых функций потерь для решения задачи многоклассовой классификации [19], [20], [21]. В качестве алгоритма настройки параметров нейронной сети был выбран стохастический метод оптимизации Adam. Модель обучалась на 50 эпохах на датасете разделенном на батчи размером 1024 элементов.

Результаты обучения представлены на рис. 13, рис. 14, рис. 15:

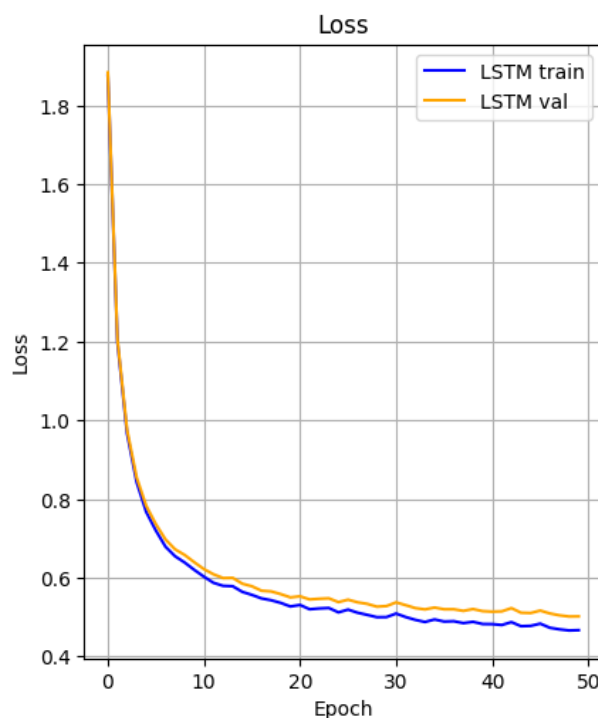


Рисунок 13 – График значений функции потерь от номера эпохи спроектированной нейронной сети

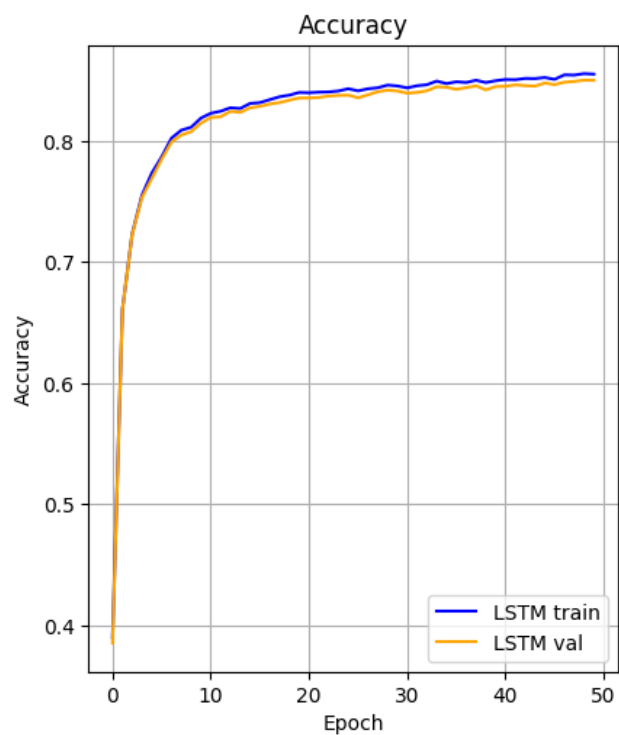


Рисунок 14 – График точности от номера эпохи спроектированной нейронной сети

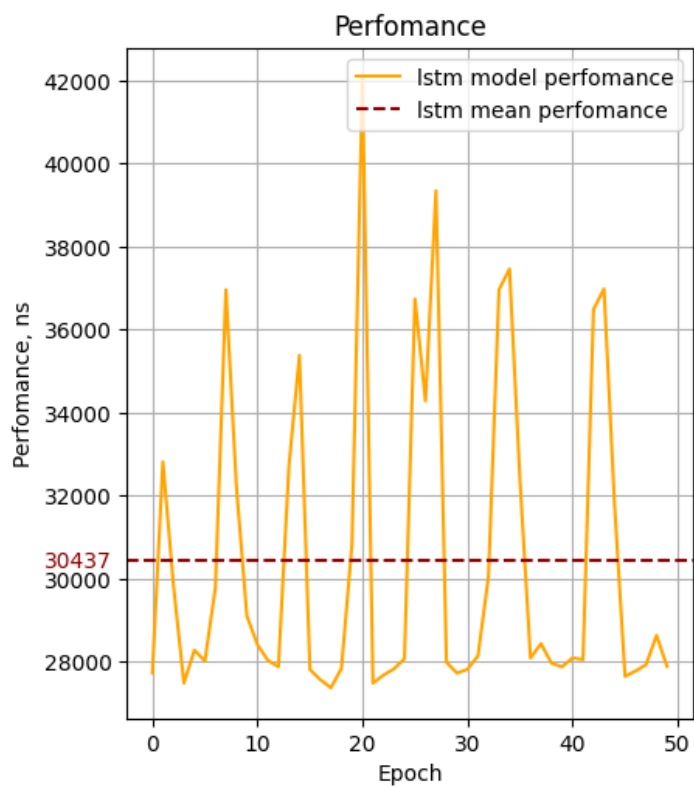


Рисунок 15 – График производительности от номера эпохи спроектированной нейронной сети

На тестовой выборке точность равна 84%, а среднее время предсказания равно 30437 наносекундам. Данная модель обладает достаточно хорошей точностью (больше 80%), но использование сложной модели, включающей слой Word Embedding, Bi-LSTM слой и линейный слой, для прогнозирования системных вызовов неэффективно из-за высокой вычислительной нагрузки. Время, требуемое на предсказание такой моделью (около 30 микросекунд), не соответствует скорости выполнения задач операционных систем. Чтобы быть эффективными в ОС, модели должны быть более простыми и быстрыми, способными быстро и эффективно предсказывать системные вызовы. Поэтому было принято решение использовать слой Word Embedding и линейный слой для предсказания.

Итоговая нейронная сеть состоит из следующих слоев:

- Word Embedding, где размерность токена равна 4;
- Линейный, на вход которого подается 5 системных вызова по 4 токена, и каждый токен является вектором размерностью 4, таким образом на вход подается 80 значений, а выход остается неизменным, он равен количеству системных вызовов в наборе данных.

Архитектура итоговой нейронной сети представлена на рис. 16:

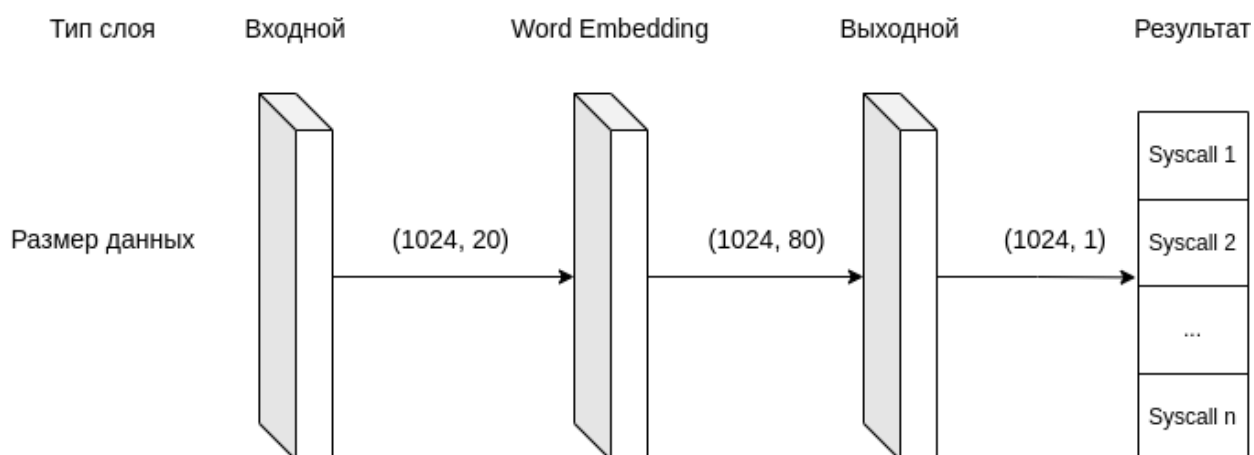


Рисунок 16 – Архитектура итоговой нейронной сети

Графики loss, точности и производительности двух нейросетей представлены на рис. 17, рис. 18, рис. 19:

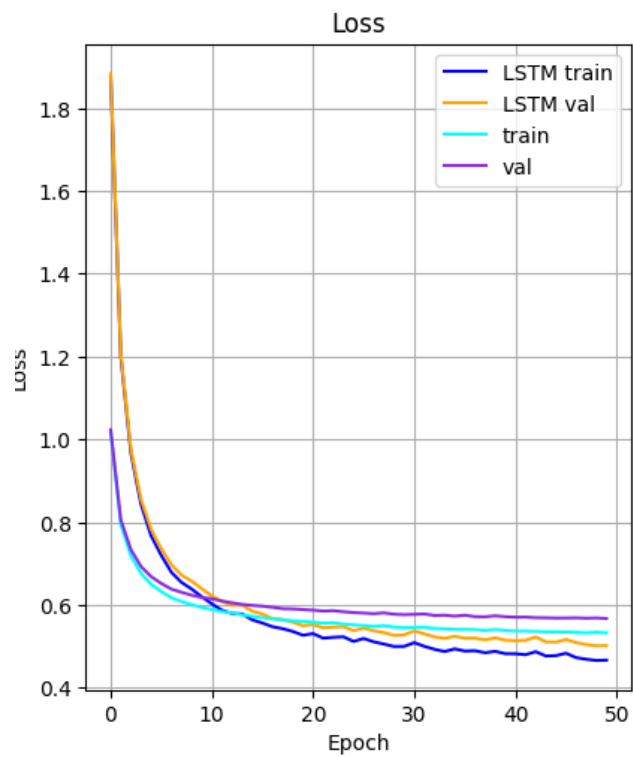


Рисунок 17 – График сравнений значений функции потерь нейронных сетей

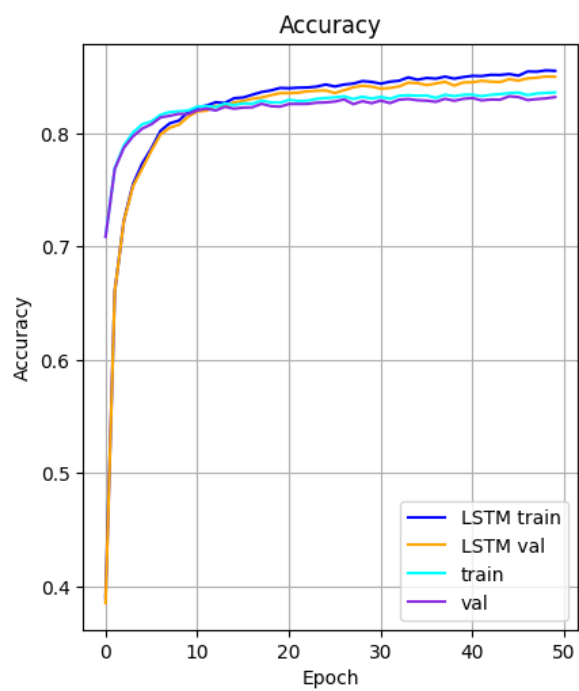


Рисунок 18 –График сравнений точностей нейронных сетей

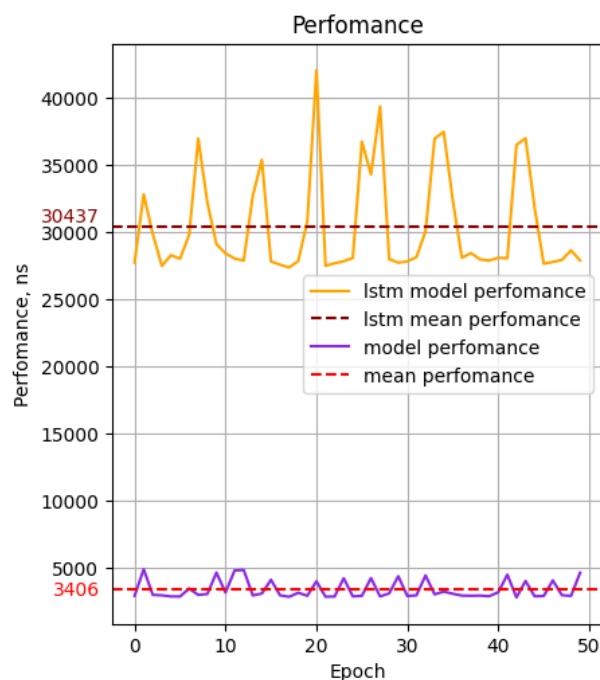


Рисунок 19 – График сравнений производительности нейронных сетей

Исходя из полученных графиков можно заметить, что у итоговой модели немного ниже точность по сравнению с нейронной сетью с Bi-LSTM слоем, но зато время для предсказания уменьшилось примерно в 10 раз.

4.3 Используемые технологии

Для создания и обучения нейронных сетей доступны различные инструменты и библиотеки, и одним из наиболее популярных языков программирования для этой цели является Python [6]. Python предлагает широкий спектр библиотек, таких как TensorFlow, PyTorch и Keras, которые представляют необходимые инструменты и функциональность для разработки и обучения нейронных сетей. Благодаря этому разнообразию средств Python является предпочтительным языком программирования для разработки и обучения нейронных сетей.

Для разработки нейронных сетей было решено использовать библиотеку PyTorch из-за её простого и гибкого интерфейса для создания и обучения сложных моделей. PyTorch обладает хорошей производительностью благодаря поддержке вычислений на GPU и различных алгоритмов оптимизации,

что делает её отличным выбором для исследований в области машинного обучения. Кроме того, наличие обширного сообщества пользователей обеспечивает доступ к дополнительным библиотекам и инструментам для улучшения работы с нейронными сетями [24].

Выбрана онлайн среда разработки Google colab по следующим причинам:

- Цена: Google Colab предоставляет бесплатные вычислительные ресурсы;
- Гибкость: Запуск текста программы, не требуя установки дополнительного ПО, на ЯП Python прямо в браузере, что делает его удобным для использования на любом устройстве;
- Облачные вычисления: Google colab работает на серверах Google, что позволяет выполнять вычисления на мощных вычислительных машинах.

Strace является широко используемым и активно поддерживаемым сообществом разработчиков инструментом для трассировки системных вызовов в операционной системе Linux. Он предоставляет простой и удобный интерфейс для выполнения трассировки, а также широкий набор функций и опций, которые позволяют пользователю настраивать и управлять процессом трассировки в соответствии с его потребностями.

5 БЕЗОПАСНОСТЬ ЖИЗНЕДЕЯТЕЛЬНОСТИ

С появлением новых технологий и развитием науки все большее значение приобретает вопрос обеспечения безопасности людей при выполнении ими своих профессиональных обязанностей. Именно поэтому была создана и постоянно совершенствуется наука, изучающая проблемы безопасности труда и жизнедеятельности человека.

Безопасность жизнедеятельности (БЖД) – это набор мероприятий, направленных на защиту человека в его обычной среде обитания, поддержание его здоровья и разработку методов защиты от вредных и опасных факторов. Целью БЖД является минимизация рисков возникновения травм, несчастных случаев и болезней на рабочем месте или в любой другой сфере человеческой деятельности.

В современное время компьютерная техника широко используется в различных сферах человеческой деятельности. При работе с ЭВМ человек подвергается воздействию опасных и вредных производственных факторов [23], таких как электромагнитные поля, инфракрасное и ионизирующее излучения, шум, вибрация, статическое электричество. Работа за компьютером может быть связана с серьезным умственным и эмоциональным стрессом, а также с высокой нагрузкой на органы зрения. Это требует особого внимания к вопросам здоровья и безопасности при работе с электронной вычислительной машиной.

Для устранения негативных последствий работы с компьютером необходимо соблюдать правильный режим труда и отдыха. В противном случае сотрудники могут столкнуться с проблемами зрения, головными болями, раздражительностью, нарушениями сна, усталостью и болезненными ощущениями в глазах, пояснице, шее и руках [23].

5.1 Требования к производственным помещениям

Согласно СП 2.2.3670-20 [38], для помещения, где используется вычислительная техника, необходимо соблюдать следующие требования:

- Оконные проемы должны позволять регулировать параметры световой среды в помещении;
- Площадь на одно рабочее место пользователей ПЭВМ проводящего за компьютером более четырех часов в день, если компьютер снабжен жидкокристаллическим или плазменным монитором, должна быть не менее 4,5 кв. м.

Оптимальными параметрами микроклимата [38] в помещении с компьютерами считаются:

- Температура воздуха – от 22 до 24°;
- Относительная влажность – от 15 до 75%;
- Скорость движения воздуха – не более 0,1 м/с.

5.2 Требования к рабочему месту сотрудника

В производственных помещениях при выполнении основных или вспомогательных работ с использованием ЭВМ, согласно СанПиН 1.2.3685-21 [25], ГОСТ Р ИСО 9241-4-2009 [36], ГОСТ Р ИСО 9241-5-2009 [37] должны выполняться следующие требования:

- Уровни шума на рабочих местах не должны превышать предельно допустимых значений [25];
- Освещенность на рабочем месте должна составлять от 300 до 500 лк, без бликов на экране. Освещенность экрана не должна превышать 200 лк. [25];
- Коэффициент пульсации не должен превышать 5% [25];
- Клавиатура должна быть устойчивой во время ее использования на горизонтальной плоской поверхности, она не должна скользить или качаться [36];
- Угол наклона клавиатуры должен быть регулируемым [36];

- Экран видеомонитора должен находиться от глаз пользователя на расстоянии 450-750 мм, оптимальным расстоянием составляет 600 мм. [37];
- Для минимизации зеркальные отражения покрытие рабочей поверхности должно соответствовать состоянию матового блеска [37];
- На рабочей поверхности и ее опорном каркасе не должно быть никаких острых кромок или углов, которые могли бы причинить ущерб пользователям или вызвать чувство дискомфорта при работе [37];
- Минимальный допустимый радиус закруглений на краях и углах рабочей поверхности должен составлять 2 мм. [37];
- Рабочая поверхность, нагруженная запланированным для использования оборудованием, не должна опрокидываться, если человек опирается на какой-либо ее край или сидит на ее краю [37];
- Рекомендуется применение роликов для ножек рабочих кресел для легкого и безопасного передвижения в пределах рабочей зоны [37];
- Шарниры кресла должны обеспечивать пользователю легкое и безопасное вращение, не изгибая позвоночник и не поворачивая туловища [37];
- Клавиатуру следует располагать на поверхности стола на расстоянии не менее 100 мм от края, обращенного к пользователю [37].

5.3 Эргономика интерактивной системы

Эргономика – это наука, изучающая взаимодействие человека с окружающей средой с целью создания комфортных и удобных условий для работы и жизни. Эргономика занимается проектированием рабочих мест, оборудова-

ния, предметов быта и компьютерных программ для обеспечения безопасности и эффективности труда работника, учитывая физические и психические особенности человеческого организма.

Интерфейс пользователя IDE оказывает влияние на психофизиологические аспекты, скорость и эффективность работы. Согласно ГОСТ Р ИСО 9241-110-2016 [26] IDE PyCharm удовлетворяет семи основным принципам организации диалога пользователь-система:

1. IDE PyCharm соответствует принципу приемлемости организации диалога для выполнения производственного задания, поскольку в своем составе имеет большое количество инструментов и возможностей для эффективной работы разработчиков. Среди наиболее распространенных инструментов можно выделить следующие:
 - автоматическое дополнение текста программы, которое позволяет ускорить набор и уменьшить количество опечаток;
 - рефакторинг текста программы, которое позволяет улучшать структуру и читаемость текста программы;
 - отладка текста программы, которая помогает выявлять и исправлять ошибки;
 - интеграция с системами контроля версий, такими как Git, SVN и Mercurial, которая облегчает совместную работу над проектом.
2. IDE PyCharm соответствует принципу информативности, поскольку предоставляет разработчикам необходимую информацию о тексте программы, предупреждения и ошибки. Кроме того, IDE PyCharm предоставляет возможность настройки интерфейса и горячих клавиш, что позволяет пользователям адаптировать среду разработки под свои потребности и предпочтения, а также имеет систему подсказок к каждому элементу интерфейса и систему уведомлений разработчика.

3. IDE PyCharm соответствует принципу соответствия ожиданиям пользователей, поскольку предоставляет гибко настраиваемый интерфейс, возможность интеграции с другими инструментами и сервисами, такими как системы контроля версий, базы данных и серверы приложений, а также имеет обширную документацию и активное сообщество пользователей, которое помогает решать возникающие вопросы и проблемы.
4. IDE PyCharm соответствует принципу пригодности для обучения, так как предоставляет доступ к большому количеству учебных ресурсов, включая видео-уроки, статьи и документацию, которые помогают обучающимся улучшить свои навыки программирования на Python. Кроме того, IDE PyCharm предоставляет возможность запуска и отладки текста программы прямо из среды разработки, что позволяет видеть результаты своей работы немедленно, а также имеет встроенный редактор текста программы с подсветкой синтаксиса, автодополнением и проверкой ошибок, что облегчает написание и отладку текста программы.
5. IDE PyCharm соответствует принципу контролируемости, так как предоставляет разработчикам инструменты для мониторинга и управления процессом разработки. Среди наиболее часто используемых инструментов можно выделить встроенную систему контроля версий, возможность настройки параметров выполнения текста программы, которая позволяет пользователям контролировать поведение программы во время выполнения. Кроме того, IDE PyCharm имеет мощные средства отладки, которые помогают пользователям находить и исправлять ошибки в тексте программы, а также хранит историю выполненных команд и имеет возможность отменить или вернуть последнее выполненное действие.

6. IDE PyCharm соответствует принципу устойчивости к ошибкам, так как предоставляет разработчикам инструменты для обнаружения и исправления ошибок в тексте программы. К таким инструментам относится встроенный статический анализатор текста программы, который может выявить потенциальные проблемы в тексте программы во время его написания, возможность запуска проверки текста программы, которая помогает выявить ошибки. Кроме того, среда разработки имеет мощные средства отладки, которые помогают разработчикам быстро диагностировать и исправлять ошибки в тексте программы, а также имеет подсветку некорректных участков текста программы.
7. IDE PyCharm соответствует принципу адаптируемости к индивидуальным особенностям пользователя, так как имеет широкие возможности по настройке и персонализации интерфейса. К таким возможностям относится настройка цветовой схемы и горячих клавиш клавиатуры, выбор шрифтов и размеров текста. Кроме того, предоставляет возможность расширения функционала за счет установки дополнительных плагинов, что позволяет пользователям добавлять новые возможности и инструменты необходимые для работы.

Таким образом, IDE PyCharm имеет интерфейс, соответствующим основным семи принципам, представленным в стандарте ГОСТ Р ИСО 9241-110-2016 [26].

5.4 Оценка используемого ПО

Проведена оценка используемого программного обеспечения IDE PyCharm по ГОСТ Р ИСО/МЭК ТО 12182–2002 [35]. Характеристики используемого ПО представлены в таблице 2:

Таблица 2 – Таблица характеристик используемого ПО

Программное обеспечение	PyCharm
-------------------------	---------

Вид	Класс
Функция ПС	Разработка программ
Режим эксплуатации	Совмещенная обработка данных
Прикладная область информационной системы	Разработка программного обеспечения на языке Python

5.5 Выводы

В соответствии с СП 2.2.3670-20 [38] были сформулированы требования к производственным помещениям. Согласно СанПиН 1.2.3685-21 [25], ГОСТ Р ИСО 9241-4-2009 [36], ГОСТ Р ИСО 9241-5-2009 [37] были сформулированы требования к рабочему месту сотрудника. Была изучена эргономика интерактивной системы IDE PyCharm. Интерфейс IDE PyCharm удовлетворяет семи основным принципам ГОСТ Р ИСО 9241-110-2016 [26] и уменьшает влияние на психофизиологические аспекты пользователя. Также была проведена оценка используемого программного обеспечения IDE PyCharm по ГОСТ Р ИСО/МЭК ТО 12182–2002 [35] (см. таблицу 2).

ЗАКЛЮЧЕНИЕ

В ходе выполнения работы была достигнута поставленная цель – разработка модели искусственного интеллекта для прогнозирования действий программ в реальном времени, предназначенная для улучшения производительности операционных систем.

Для достижения поставленной цели были выполнены следующие задачи:

- Изучены существующие решения для предсказания последующего системного вызова;
- Составление списка требований к решению;
- Трассировка списка процессов. Этот шаг включает в себя получение списка последовательных событий (системных вызовов) с помощью утилиты `strace`;
- Обработка списка системных вызовов. На этом шаге произведено разбиение данных на токены, полученные данные разделены на обучающую, валидационную и тестовую выборки. Таким образом были подготовлены данные для обучения модели ИИ;
- Обзор существующих моделей искусственного интеллекта для прогнозирования системных вызовов;
- Обучение модели на обучающем наборе данных;
- Оценка качества модели ИИ на тестовой выборке;

В данной работе произведен обзор моделей искусственного интеллекта для прогнозирования последующего системного вызова в операционной системе Linux. Для проведения обзора было рассмотрено четыре модели: CNN, GRU, LSTM и Bi-LSTM. В ходе анализа выбранных моделей нейронная сеть Bi-LSTM является более эффективным методом прогнозирования по сравнению с другими методами.

Тестирование выбранной нейронной сети со слоями Word Embedding, Bi-LSTM и линейного показало, что нейронная сеть имеет недостаточно хорошее время предсказания, то есть время предсказания превышает допустимое

время в работе ОС. В связи с этим было принято решение упростить модель, используя только 2 слоя: Word Embedding и линейный. Упрощенная модель показала незначительное снижение точности (1%), но при этом скорость прогнозирования уменьшилась приблизительно в 10 раз, что стало сопоставимым со временем работы в ОС.

Для дальнейшего улучшения точности и скорости прогнозирования поведения программ в операционных системах, предлагается настраивать параметры нейронной сети, исследовать возможность использования различных видов нейронных сетей, а также вместо прогнозирования названия следующего системного вызова генерировать последовательность системных вызовов вместе с их аргументами и результатами. Кроме того, целесообразно изучить возможность использования разработанной модели для прогнозирования других типов аномалий в операционных системах.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Duesterwald E., Cascaval C., Dwarkadas S. Characterizing and predicting program behavior and its variability //2003 12th International Conference on Parallel Architectures and Compilation Techniques. – IEEE, 2003. – С. 220-231.
2. Sarikaya R., Buyuktosunoglu A. A unified prediction method for predicting program behavior //IEEE Transactions on Computers. – 2009. – Т. 59. – №. 2. – С. 272-282.
3. Minaee S. et al. Deep learning--based text classification: a comprehensive review //ACM computing surveys (CSUR). – 2021. – Т. 54. – №. 3. – С. 1-40.
4. Rama-Maneiro E., Vidal J. C., Lama M. Deep learning for predictive business process monitoring: Review and benchmark //IEEE Transactions on Services Computing. – 2021. – Т. 16. – №. 1. – С. 739-756.
5. Xu C. et al. Cache contention and application performance prediction for multi-core systems //2010 IEEE International Symposium on Performance Analysis of Systems & Software (ISPASS). – IEEE, 2010. – С. 76-86.
6. Thaker N., Shukla A. Python as multi paradigm programming language //International Journal of Computer Applications. – 2020. – Т. 177. – №. 31. – С. 38-42.
7. <https://gs.statcounter.com/os-market-share/desktop/worldwide>
8. Tsafrir D. The context-switch overhead inflicted by hardware interrupts (and the enigma of do-nothing loops) //Proceedings of the 2007 workshop on Experimental computer science. – 2007. – С. 4-es.
9. Gers F. A., Schmidhuber J., Cummins F. Learning to forget: Continual prediction with LSTM //Neural computation. – 2000. – Т. 12. – №. 10. – С. 2451-2471.
10. Hochreiter S., Schmidhuber J. LSTM can solve hard long time lag problems //Advances in neural information processing systems. – 1996. – Т. 9.
11. Hochreiter S., Schmidhuber J. Long short-term memory //Neural computation. – 1997. – Т. 9. – №. 8. – С. 1735-1780.

12. Gers F. A., Schraudolph N. N., Schmidhuber J. Learning precise timing with LSTM recurrent networks //Journal of machine learning research. – 2002. – T. 3. – №. Aug. – C. 115-143.
13. Huang Z., Xu W., Yu K. Bidirectional LSTM-CRF models for sequence tagging //arXiv preprint arXiv:1508.01991. – 2015.
14. Yu Y. et al. A review of recurrent neural networks: LSTM cells and network architectures //Neural computation. – 2019. – T. 31. – №. 7. – C. 1235-1270.
15. Staudemeyer R. C., Morris E. R. Understanding LSTM--a tutorial into long short-term memory recurrent neural networks //arXiv preprint arXiv:1909.09586. – 2019.
16. Shimodaira H. Single Layer Neural Networks //Learning and Data Note. – 2015.
17. BB S. D. S. P. D., Varshapriya M. P., Ambavkar P. P. P. DEBUGGING ON LINUX.
18. Subramanian V. Deep Learning with PyTorch: A practical approach to building neural network models using PyTorch. – Packt Publishing Ltd, 2018.
19. Sergeevna Bosman A., Engelbrecht A., Helbig M. Visualising Basins of Attraction for the Cross-Entropy and the Squared Error Neural Network Loss Functions //arXiv e-prints. – 2019. – C. ArXiv: 1901.02302.
20. Golik P., Doetsch P., Ney H. Cross-entropy vs. squared error training: a theoretical and experimental comparison //Interspeech. – 2013. – T. 13. – C. 1756-1760.
21. Glorot X., Bengio Y. Understanding the difficulty of training deep feedforward neural networks //Proceedings of the thirteenth international conference on artificial intelligence and statistics. – JMLR Workshop and Conference Proceedings, 2010. – C. 249-256.
22. Allen C., Hospedales T. Analogies explained: Towards understanding word embeddings //International Conference on Machine Learning. – PMLR, 2019. – C. 223-231.

23. Белехов Александр Николаевич Научно-технический прогресс и безопасность труда // Инновации и инвестиции. 2016. №3. URL: <https://cyberleninka.ru/article/n/nauchno-tehnicheskij-progress-i-bezopasnost-truda> (дата обращения: 12.05.2024).
24. Steiner B. et al. Pytorch: An imperative style, high-performance deep learning library. – 2019.
25. СанПиН 1.2.3685-21
26. ГОСТ Р ИСО 9241-110-2016
27. Di Mauro N., Appice A., Basile T. M. A. Activity prediction of business process instances with inception CNN models //AI* IA 2019–Advances in Artificial Intelligence: XVIIIth International Conference of the Italian Association for Artificial Intelligence, Rende, Italy, November 19–22, 2019, Proceedings 18. – Springer International Publishing, 2019. – С. 348-361.
28. Tax N. et al. Predictive business process monitoring with LSTM neural networks //Advanced Information Systems Engineering: 29th International Conference, CAiSE 2017, Essen, Germany, June 12-16, 2017, Proceedings 29. – Springer International Publishing, 2017. – С. 477-492.
29. Hinkka M. et al. Classifying process instances using recurrent neural networks //Business Process Management Workshops: BPM 2018 International Workshops, Sydney, NSW, Australia, September 9-14, 2018, Revised Papers 16. – Springer International Publishing, 2019. – С. 313-324.
30. Mohd N., Singhdev H., Upadhyay D. TEXT CLASSIFICATION USING CNN AND CNN-LSTM //Webology. – 2021. – Т. 18. – №. 4. – С. 2440-2446.
31. Yin W. et al. Comparative study of CNN and RNN for natural language processing //arXiv preprint arXiv:1702.01923. – 2017.
32. Hameed Z., Garcia-Zapirain B. Sentiment classification using a single-layered BiLSTM model //Ieee Access. – 2020. – Т. 8. – С. 73992-74001.

33. Dey R., Salem F. M. Gate-variants of gated recurrent unit (GRU) neural networks //2017 IEEE 60th international midwest symposium on circuits and systems (MWSCAS). – IEEE, 2017. – С. 1597-1600.
34. Irie K. et al. LSTM, GRU, highway and a bit of attention: An empirical overview for language modeling in speech recognition //Interspeech. – 2016. – С. 3519-3523.
35. ГОСТ Р ИСО/МЭК ТО 12182–2002
36. ГОСТ Р ИСО 9241-4-2009
37. ГОСТ Р ИСО 9241-5-2009
38. СП 2.2.3670-20
39. Fox R. Linux with operating system concepts. – Chapman and Hall/CRC, 2021.
40. Luke K. Interaction Between the User and Kernel Space in Linux. – 2017.